INTRODUCTION

# CacheWise replication and KV cache recovery

Replicating CacheWise and evaluating host-DRAM recovery

CENTRAL QUESTION

## How can coding agents reuse previous computation when GPU memory is limited?

# What do the CacheWise traces contain?

## CATraces: real coding sessions

Claude Code interaction logs
Model requests + tool calls
Timing + prompt / output lengths

Released content is redacted; replay
preserves lengths and prefix growth.

## Reuse across turns

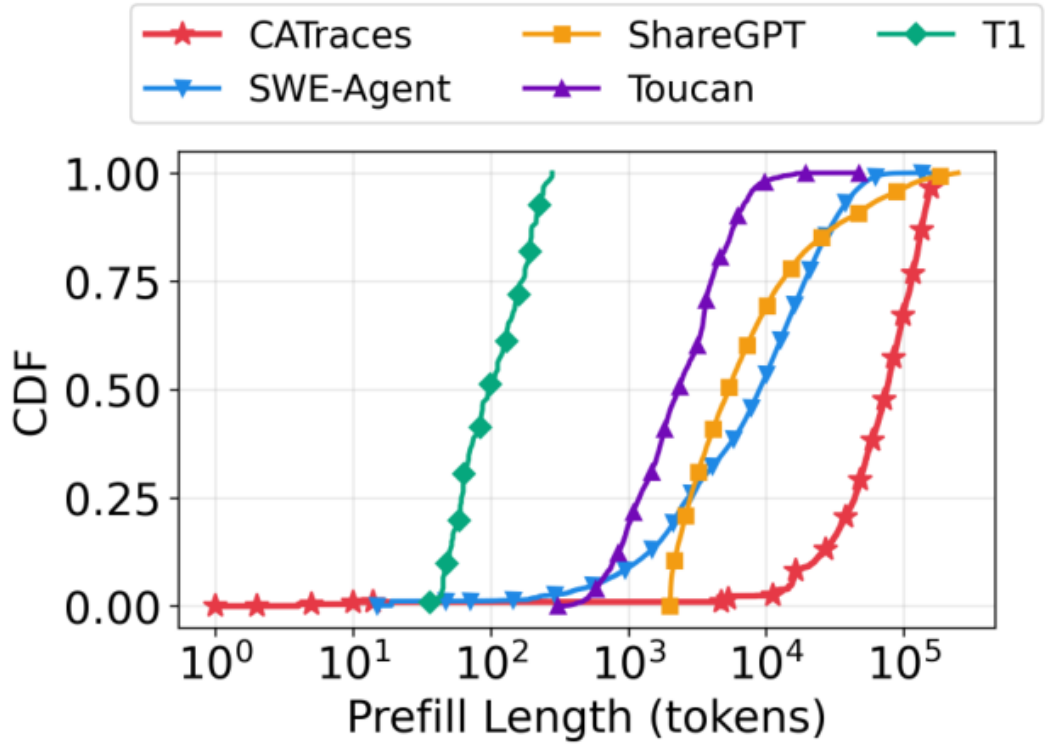
Growing, shared prompt prefixes
Long inputs with short outputs
Tool waits between requests

Requests arrive in sessions,
with repeated opportunities for reuse.

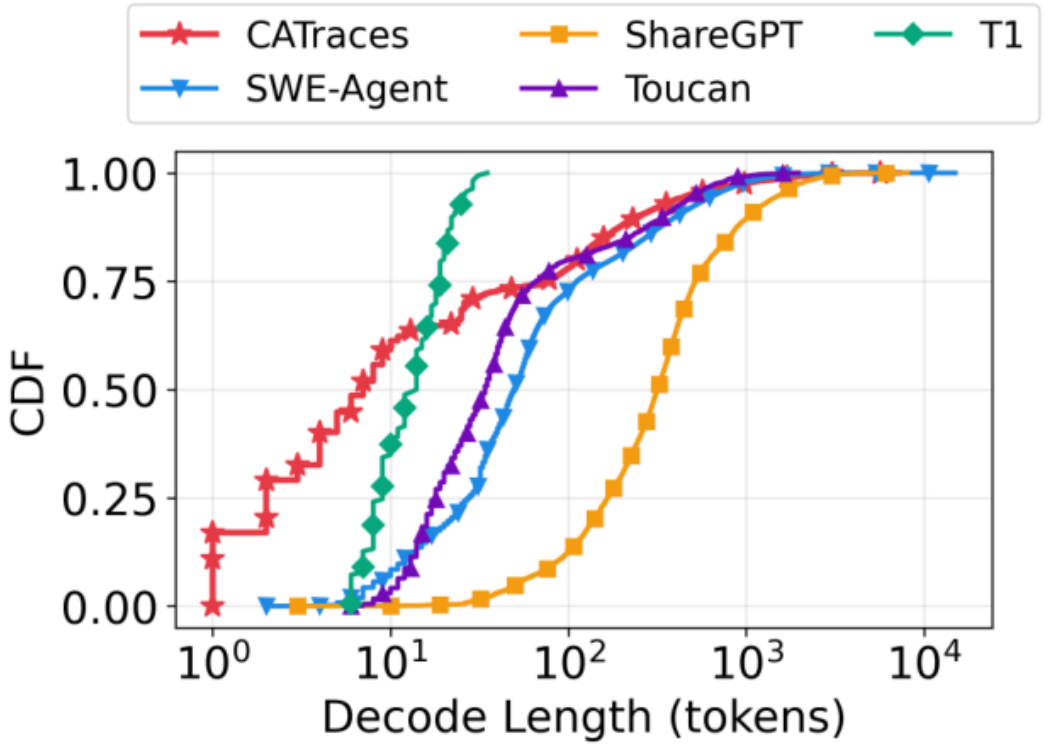
## Why prefix-aware scheduling can help

Reuse a resident prefix → less new prefill → fewer additional KV blocks.
A smaller active working set can reduce eviction pressure.

# CATraces: long inputs, short outputs




(a) Prefill length distributions for different datasets




(b) Decode length distributions for different datasets

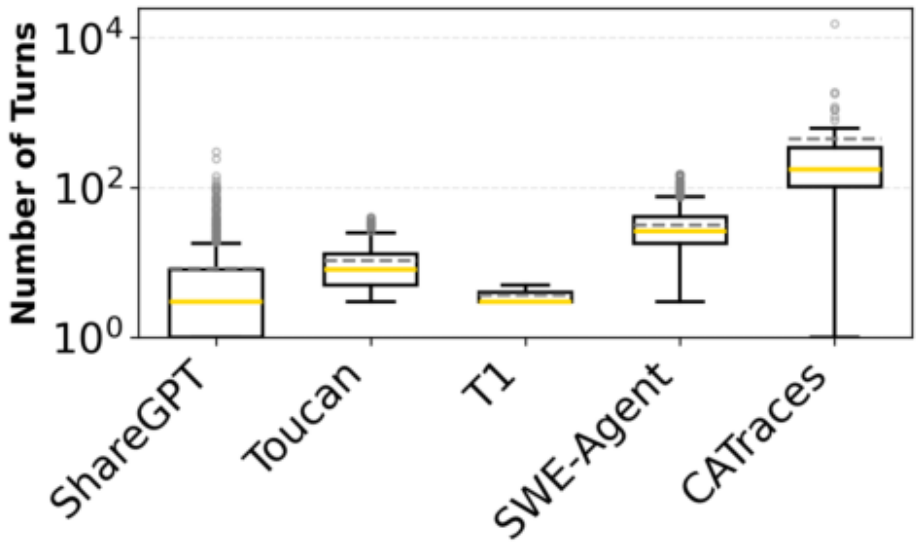
**CATraces (red) vs. ShareGPT chat (orange): longer prefill, shorter decode.**

CDF = fraction of requests at or below the token length. Published results, not our replay.

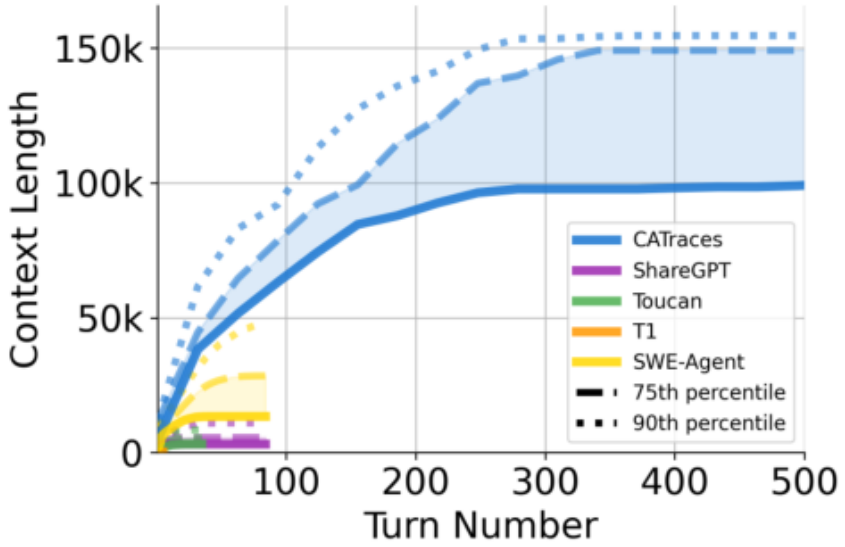
Cropped from Tiwari et al., CacheWise v1, Fig. 2 (PDF p. 4); CC BY 4.0

# CATraces differ from traditional chat

## More turns per session (Fig. 3)




## Growing context across turns (Fig. 6)




One task drives many model/tool turns; growing KV prefixes compete for memory.
Session completion matters alongside individual request latency.

Cropped from Tiwari et al., CacheWise v1, Figs. 3 and 6 (PDF p. 4); CC BY 4.0

# CacheWise predicts when a session returns

## Schedule requests

**Additional KV blocks = required − resident**

Serve the smallest additional-block cost first.

Uses current GPU prefix overlap.

## Predict duration → order evictions

**Tool name + arguments** → **TF–IDF → trained KMeans Historical duration samples**

**Condition on the tool still running:**

$$r = E[D - e \mid D > e]$$

Evict unreferenced KV with largest r; refresh predictions as time passes.

D = total duration; e = elapsed time. Replication: tool names only (arguments redacted).

Algorithm adapted from CacheWise v1 §§5.1–5.3; original-paper argument-aware predictor

# CacheWise combines scheduling and eviction

| Scheduler (rows) | Eviction policy (columns) | |
|---|---|---|
| | **LRU** | **Predictive** |
| **FCFS** | **Baseline**<br>**FCFS + LRU** | **Eviction only**<br>**FCFS + predictive** |
| **Prefix-aware** | **Scheduling only**<br>**Prefix + LRU** | **CacheWise (both)**<br>**Prefix + predictive** |

Bottom-right: adapted CacheWise uses both components.

## Testbed

**Qwen2.5-Coder-14B-Instruct**

1 × RTX PRO 6000 Blackwell Max-Q GPU

BF16 · tensor parallelism 1

KV: 287,696 tokens (17,981 × 16)

Prefill chunk: 512 tokens

Max sequence length: 131,072

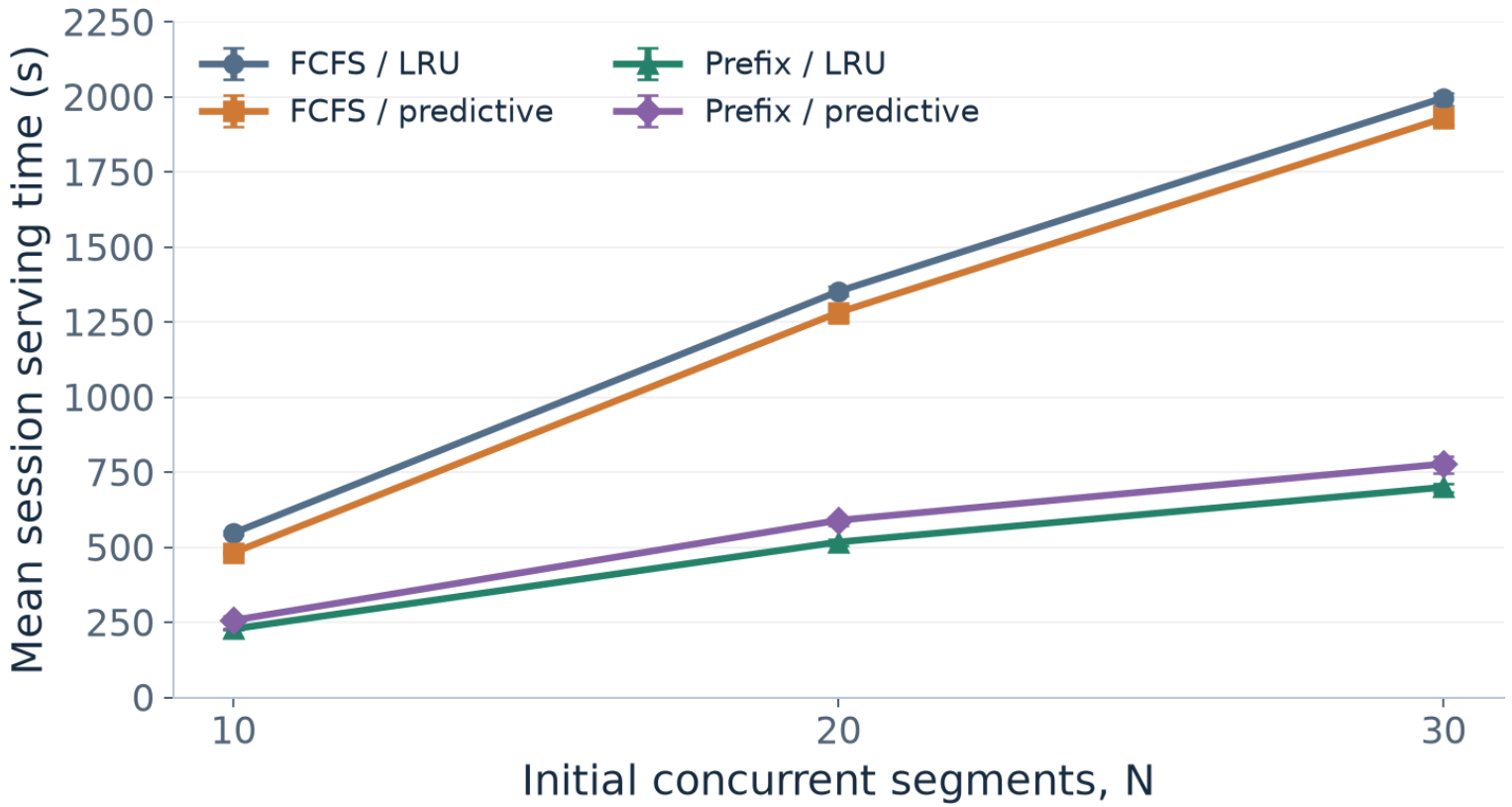
Max scheduled sequences: 32

**Coverage: 5 settings × 4 policies × 3 repeats = 60 runs · 6,384 requests**

Metric: mean summed model-request latency per session, excluding tool waits.

Evidence: four-policy replication; report Tables 1–3 and §3

# Prefix / LRU has the lowest mean




**5 / 5**

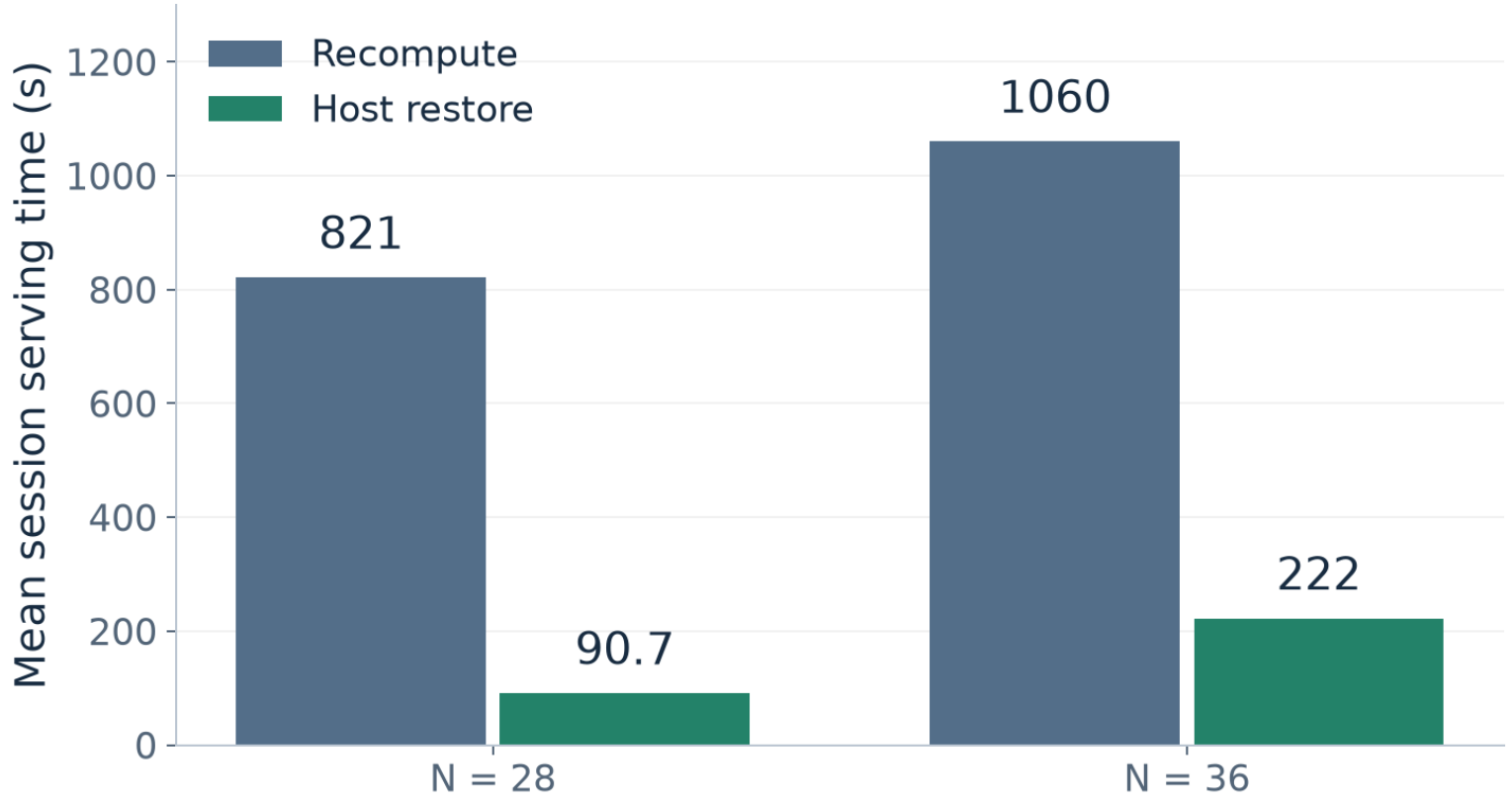
**settings favor prefix / LRU**

58–65% lower mean vs. FCFS / LRU in the load sweep

Three repeats per cell. Ranges are observed minima/maxima; lower is better.

# Host restoration reduces session time

Recovery tradeoff: save + restore + management < recomputation




**9.1× / 4.8×**

reported time ratios
at N = 28 / 36

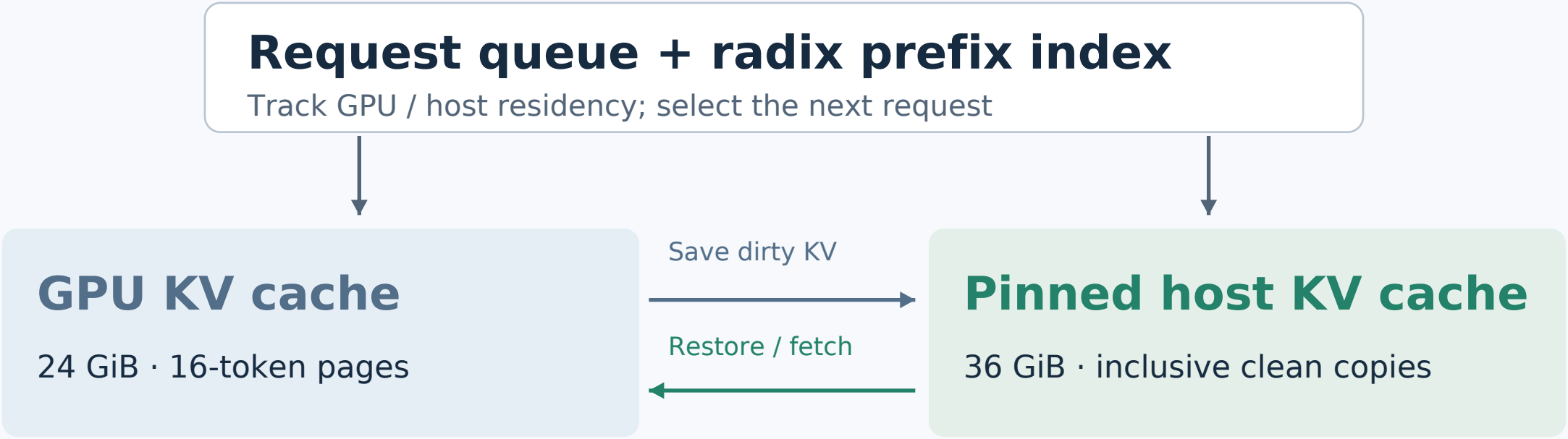
**+36 GiB host KV**
**24 GiB GPU KV**
**fixed**

One observation
per bar

Separate Mini-SGLang study. Added-capacity comparison; lower is better.

Evidence: host-dram-reported-results.json; report Figure 3a and §6

# Preserve evicted KV in host DRAM




## On demand

Restore before prefill.

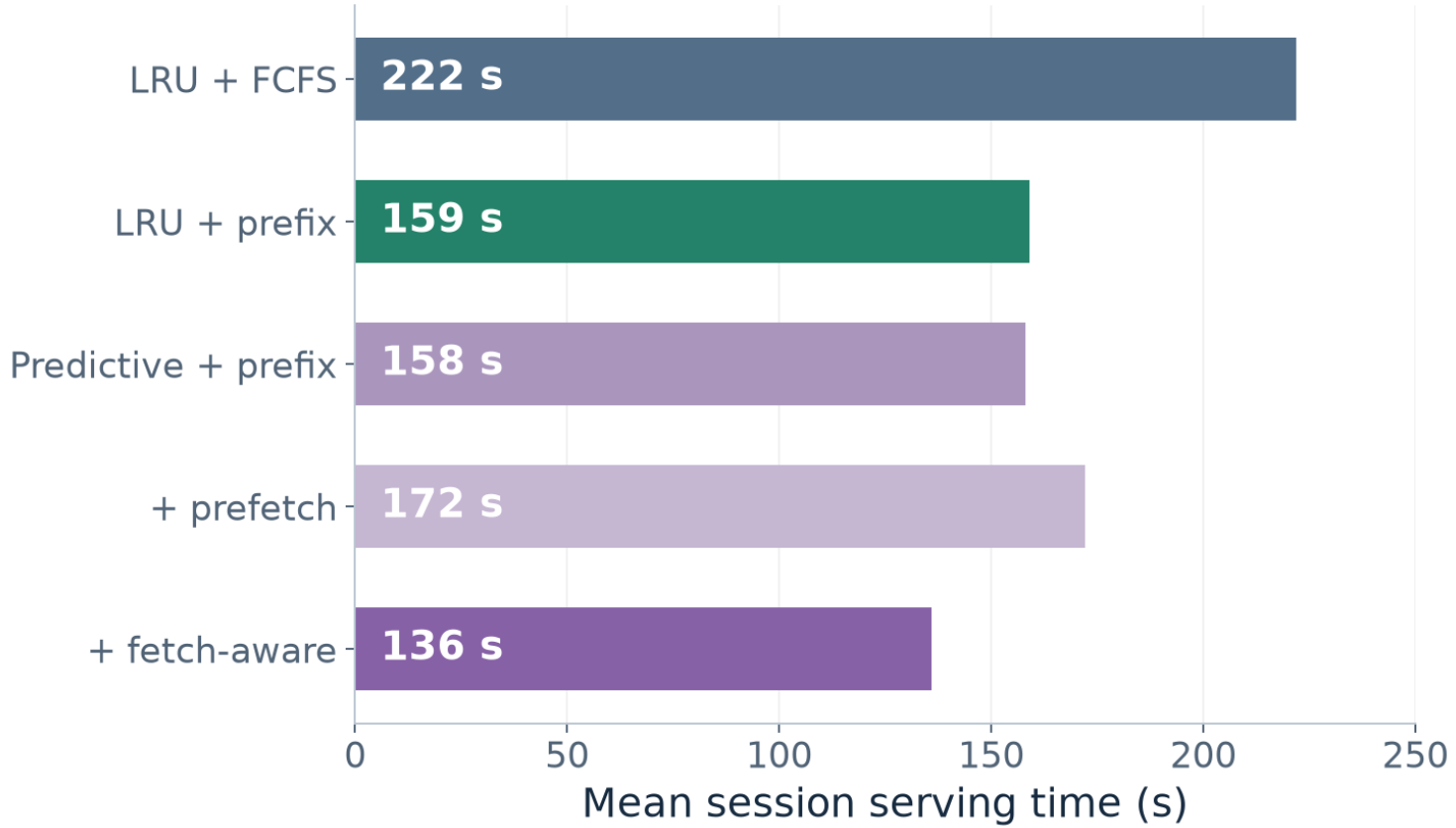
## Prefetch

Overlap predicted reuse.

## Fallback

Recompute on a miss.

# Fetch-aware ordering discounts host hits




## Priority cost

**new prefill tokens + 0.05 × host tokens**

Mean of 2 observations
166.5 s → 135.5 s

Sparse repetitions; no confidence interval.

Bars: first reported runs. Side-panel means use the two available runs.

Evidence: Mini-SGLangs 3f9aee7; report Figure 3b and Eq. (3)

# Keep the conclusions within the evidence

## Replication

Synthetic replay and selected segments.

One retained preemption; extra prefill work unmeasured.

## Host results

Separate engine and replay.

Added host capacity; most configurations run once.

## SSD extension

Not implemented or measured.

Needs full I/O, staging, metadata and recovery.

**Interpret both studies separately; retain the declared protocol deviation.**

# Three takeaways

01

## Scheduling and eviction interact.

Prefix / LRU has the lowest mean in all five replication settings.

02

## Host copies can avoid costly recovery.

Large reported gains include the added 36 GiB backing tier.

03

## Order requests by recovery cost.

Fetch-aware results are promising; SSD evaluation remains future work.